COURSE NAME

SOFTWARE ENGINEERING

CSC 3114

(UNDERGRADUATE)

CHAPTER 12

SOFTWARE TESTING

ARJUN KUMAR BOSE ARNOB
LECTURER, CS, AIUB
arnob@aiub.edu

SOFTWARE TESTING

- Testing is the process of exercising a program with the specific intent of finding errors prior to delivery to the end user
- Software testability is simply how easily [a computer program] can be tested

Testing Shows

- Error
- Requirements Conformance
- Performance
- An indication of quality

WHO TESTS THE SOFTWARE?

Developer

- Understands the system but, will test "gently“ and, is driven by "delivery“
- Experiencing the software operation (known to the developer)

Independent tester

- Must learn about the system, but, will attempt to break it and, is driven by “quality”
- Exploring the software operation (unknown to the tester)

V & V

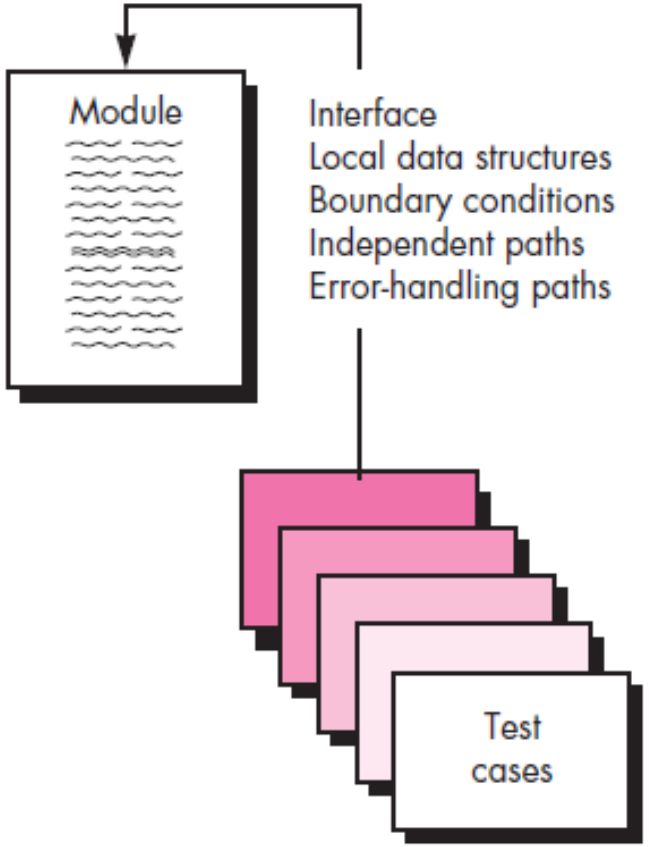
- *Validation* refers to a different set of tasks that ensure that the software that has been built is traceable to customer requirements.
- *Verification* refers to the set of tasks that ensure that software correctly implements a specific function/process.
- Boehm states this another way:
  - *Validation:* "Are we building the right product?“
  - *Verification:* "Are we building the product right?"

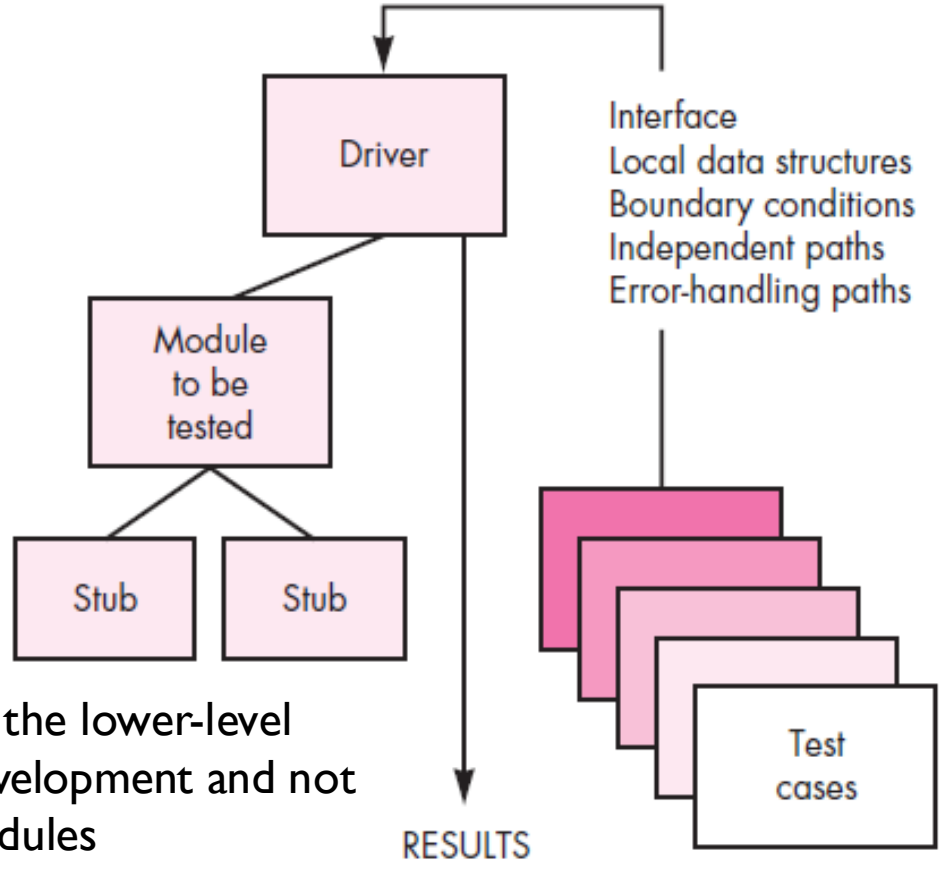
TESTING STRATEGY

- We begin by 'testing-in-the-small' and move toward 'testing-in-the-large'
- For conventional software
  - The module (component) is our initial focus
  - Integration of modules follows
- For OO software
  - Our focus when "testing in the small" changes from an individual module (the conventional view) to an OO class that encompasses attributes and operations and implies communication and collaboration

UNIT TESTING

Tests a small software unit at a time, which is typically performed by the individual programmer who implemented the unit prior to Integration testing

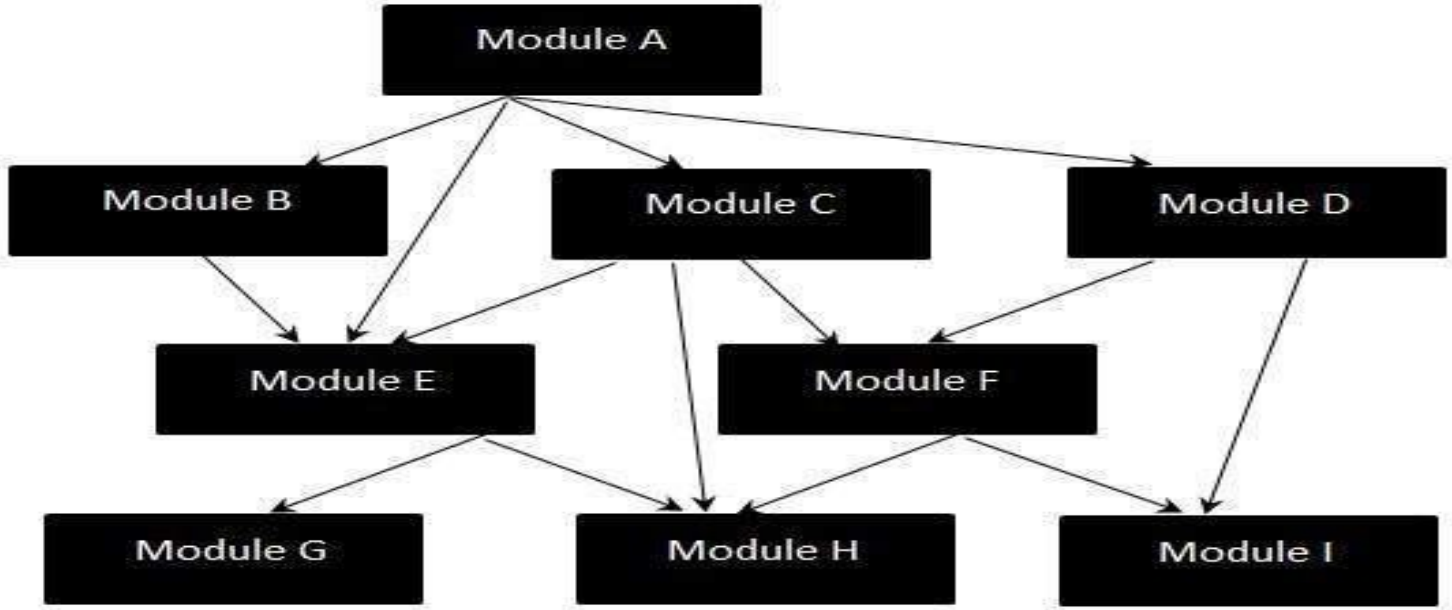




Stub: it is the behavior of the lower-level modules that are under development and not yet integrated to other modules

INTEGRATION TESTING STRATEGIES

- System integration testing (SIT) is a systematic technique for assembling a software system while conducting tests to uncover errors associated with interfacing the modules
- **the "big bang" approach:** Big Bang Integration Testing is an integration testing strategy where all units are linked at once, resulting in a complete system.



SMOKE TESTING

Smoke testing steps:

- Software components that have been translated into code are integrated into a "**daily build**"
  - Smoke testing, also known as "Build Verification Testing" is a type of software testing that is typically performed at the beginning of the development process to ensure that the most critical functions of a software application are working correctly.
  - It is used to quickly identify and fix any major issues with the software before more detailed testing is performed.
  - The goal of smoke testing is to determine whether the build is stable enough to proceed with further types of testing.

OBJECT-ORIENTED TESTING

- Class testing is the equivalent of unit testing
  - Operations within the class are tested
  - The state behavior of the class is examined

- Integration applied three different strategies
  - Thread-based testing—integrates the set of classes required to respond one input or event
  - Use Case based testing—integrates the set of classes required to respond to one use case
  - Cluster testing—integrates the set of classes required to demonstrate one collaboration

HIGHER ORDER TESTING

- **System testing:** focus is on system integration (e.g. hardware integration, OS compatibility)
- **Alpha testing** is an internal, controlled process to find bugs and technical issues within a nearly complete product, performed by developers and QA engineers in a lab setting.
- **Beta testing** is an external, real-world process that involves end-users from the target market to gather feedback on functionality, usability and overall user experience before the final release.
- **Recovery testing:** forces the software to fail in a variety of ways and verifies that recovery is properly performed
- **Security testing:** verifies that protection mechanisms built into a system will, in fact, protect it from improper penetration
- **Stress testing:** executes a system in a manner that demands resources in abnormal quantity, frequency, or volume
- **Performance Testing:** test the run-time performance of software within the context of an integrated system (e.g. time required to response a request, compliance with operational constraints)

DEBUGGING

- In many cases, the non-corresponding data are a symptom of an underlying cause as yet hidden error
- The debugging process attempts to match symptom with cause, thereby leading to error correction

- symptom may disappear when another problem is fixed
- cause may be due to a combination of non-errors
- cause may be due to a system or compiler error
- cause may be due to assumptions that everyone believes

DEBUGGING TECHNIQUES

- **Brute force testing**
  - most common; but least efficient
  - memory dumps are taken, run-time traces are invoked, and the program is loaded with output statements **(Dynamic Testing)**
- **Backtracking**
  - common debugging approach that can be used successfully in small programs
  - source code is traced backward (manually) until the cause is found
- **Cause elimination**
  - a "cause hypothesis" is devised
  - if initial tests indicate that a particular cause hypothesis shows promise, data are refined in an attempt to isolate the bug (c/a-b where the possibility of a-b is zero)

WHITE-BOX TESTING

Using white-box testing methods, you can derive test cases that

(1) guarantee that all independent paths within a module have been exercised at least once,

(2) exercise all logical decisions on their true and false sides,

(3) execute all loops at their boundaries and within their operational bounds, and

(4) exercise internal data structures to ensure their validity.

BLACK-BOX TESTING

- Focuses on the functional requirements of the software
- Black-box testing attempts to find errors in the following categories:

(1) incorrect or missing functions

(2) interface errors

(3) errors in external database access (accessibility)

(4) behavior or performance errors

(5) initialization and termination errors

TEST CASE SAMPLE

Project Name: XYZ			**Test Designed by: Member's Name**	
Test Case ID: TC_1			Test Designed date: 02/7/2025	
Test Priority (Low, Medium, High): Medium			Test Executed by: Name	
Module Name: Login Session			Test Execution date: 22/08/2025	
Test Title: Verify login with valid username and password				
Description: Test the website login page				
Precondition: The user has a valid username and password				
Dependencies: if any				
Test Steps	**Test Data**	**Expected Results**	**Actual Results**	**Status (Pass/Fail)**
1. **Go to the site** 2. **Enter username** 3. **Enter password** 4. **Click submit**	Username: urs99 Password: 321	The user should log in to the application	As expected	Pass

REFERENCES

- R.S. Pressman & Associates, Inc. (2010). *Software Engineering: A Practitioner's Approach*.
- Kelly, J. C., Sherif, J. S., & Hops, J. (1992). An analysis of defect densities found during software inspections. *Journal of Systems and Software*, *17*(2), 111-117.
- Bhandari, I., Halliday, M. J., Chaar, J., Chillarege, R., Jones, K., Atkinson, J. S., & Yonezawa, M. (1994). In-process improvement through defect data interpretation. *IBM Systems Journal*, *33*(1), 182-214.